SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

TRICHY CAMPUS | DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FULL STACK WEB DEVELOPMENT

TASK 2: INTERACTIVE JAVASCRIPT AND REACTJS APPLICATION DEVELOPMENT

CODETRACK

Interactive Coding Practice & Problem Management Workspace

STUDENT NAME	AJAYAMSAVASAN ARULMURUGAN
REGISTER NUMBER	RA2411003050055
PROGRAM / SECTION	B.Tech CSE - A
YEAR / SEMESTER	3rd Year / 5th Semester
ASSIGNMENT / PROBLEM NO.	Task 2 - Problem No. 43
DATE OF SUBMISSION	01-10-2026
COURSE	Full Stack Web Development
FACULTY COORDINATOR	Dr. P. Hariharan, AP, CSE
ACADEMIC YEAR	2026 - 2027
PROJECT DOMAIN	Technology and Creative Applications - Coding Practice Tracker
LIVE DEMO	Not deployed in this package - deployment ready

Design - Develop - Deploy

1. ABSTRACT

A new implementation of Problem No. 43 - Coding Practice Tracker

CodeTrack is a responsive ReactJS single-page application created to help a learner maintain a structured record of programming problems solved, attempted, or marked for review. Instead of treating practice as a simple checklist, the system stores each problem as a small learning record containing its topic, programming language, difficulty, status, solve time, date and personal notes. The application then turns those records into actionable summaries such as weekly output, current streak, average solve time, difficulty distribution, topic concentration and progress toward configurable goals.

The project follows the supplied Task 2 requirements by using HTML5 for semantic structure, CSS3 for responsive design, JavaScript ES6+ for data manipulation, React components for the interface, and React Hooks for state and side-effect handling. The implementation includes four client-side views: Dashboard, Problem Library, Analytics and Goals. A controlled form supports add and edit operations with validation, while the library supports search, multi-criteria filtering, bookmarking and deletion. Data is persisted with browser localStorage so that reloading the page does not erase the working dataset.

The interface is intentionally different from the supplied Habit Tracker example. The visual hierarchy uses a fixed navigation rail, a practice-signal hero section, analytical cards and a table-oriented problem library. The data model also changes completely: there is no per-date habit-progress object. Instead, CodeTrack records individual coding tasks and derives statistics from the problem history.

Keywords

ReactJS, JavaScript ES6+, coding practice, problem tracker, localStorage, CRUD, filtering, analytics, responsive UI, form validation, streaks, goals.

Assignment alignment

Requirement	CodeTrack implementation
Interactive ReactJS app	Component-driven SPA with four client-side views
HTML5 + CSS3	Semantic form controls and responsive CSS grid/flex layouts
JavaScript ES6+	map, filter, reduce, Set, template literals, spread syntax
React components + hooks	Functional components with useState, useEffect and useMemo
Form + validation	Controlled log/edit form with title/topic/time/link checks
Interactive operations	Add, edit, delete, search, filter, bookmark and goal updates
At least three views	Dashboard, Library, Analytics and Goals
Clear structure	index.html, src/main.jsx, src/App.jsx and src/styles.css

2. INTRODUCTION AND PROBLEM STATEMENT

2.1 Background

Coding practice generates information that is easy to lose: which problems were solved, which topics keep causing difficulty, how much time a problem required, and which questions should be revisited. A plain notes document can store these details but does not automatically turn them into useful progress indicators. A specialized practice workspace can bridge that gap without requiring a backend for a classroom-scale application.

2.2 Problem statement

Design and implement an interactive ReactJS application that records coding problems, captures their difficulty, topic, language, status and time, supports searching and filtering, and presents calculated practice analytics and goals through responsive client-side views.

2.3 Why this is a different project

- Domain shift: the supplied report tracks personal habits; CodeTrack tracks programming problems and learning evidence.
- Data shift: the supplied report uses daily progress records, while CodeTrack uses one record per coding problem plus goal settings.
- Navigation shift: CodeTrack uses a left navigation rail with Dashboard, Library, Analytics and Goals rather than Today, Week and Manage.
- Interaction shift: CodeTrack adds topic/language/difficulty/status filters, bookmarks, JSON backup export, theme switching and configurable practice targets.
- Analytical shift: metrics focus on solved output, solve-time trends, difficulty mix and topic clusters instead of habit completion percentages.
- Content shift: the report text, interface labels, records, workflow descriptions and implementation are newly written for this project.

2.4 Target users

- Computer science students preparing for coding interviews and laboratory evaluations
- Self-learners maintaining a small DSA or programming practice archive
- Front-end learners studying React state, derived data, forms and responsive UI

3. OBJECTIVES, SCOPE AND KEY DIFFERENTIATORS

3.1 Objectives

1. Build a responsive ReactJS workspace for coding-problem records.
2. Provide CRUD operations for problem entries without page reloads.
3. Use controlled forms and client-side validation for clean data entry.
4. Provide search and filters for title, topic, language, difficulty, status and bookmarks.
5. Calculate practice indicators including weekly solved count, average solve time and consecutive-day streak.
6. Visualize difficulty distribution and topic concentration using lightweight dashboard charts.
7. Allow the learner to configure monthly solved targets and weekly practice-time/topic targets.
8. Persist records and goal values in browser localStorage and provide a JSON backup export.
9. Keep the component structure understandable enough for a student to explain during evaluation.

3.2 Scope

In scope	Out of scope
Single-user browser application	Multi-user accounts and permissions
Coding problem metadata and notes	Server-side database
Local persistence	Cloud synchronization
Client-side analytics	Machine-learning recommendations
Responsive web UI	Native Android/iOS application
JSON export backup	Encrypted cloud backup service

3.3 Key differentiators

Area	CodeTrack design decision
Visual language	Editorial dashboard with a navigation rail, metric cards and analytical panels
Primary record	A coding problem with topic, language, difficulty, status, time and notes
Views	Dashboard / Problem Library / Analytics / Goals
Progress model	Counts, time totals, topic coverage and streak derived from solved records
Persistence	localStorage for problems + goal settings
Extra utilities	Bookmarks, JSON export, theme toggle, review/attempted status

4. TECHNOLOGIES AND SYSTEM ARCHITECTURE

4.1 Technology stack

Technology	Purpose in CodeTrack
HTML5	Document host, accessible form labels and semantic structure
CSS3	Responsive layout, custom properties, cards, tables, modal and theme styling
JavaScript ES6+	Data transforms, validation, date operations and export logic
ReactJS	Component-driven declarative UI
React Hooks	useState for state, useEffect for persistence/theme side effects, useMemo for derived data
Browser localStorage	Client-side persistence without a server
Vite (development)	Local React development and production build pipeline
Git / GitHub (deployment target)	Version control and static-host deployment path

4.2 Architecture

CodeTrack follows a lightweight three-layer client architecture. The presentation layer contains the sidebar, top bar, cards, table, charts, goal panels and modal form. The logic layer contains state updates, filtering, derived statistics, validation and export operations. The persistence boundary uses localStorage as a browser-local data store. The user can therefore close and reopen the application without losing the stored problem set on the same device/browser.

PRESENTATION LAYER	LOGIC / STATE LAYER	PERSISTENCE LAYER
React functional components	useState / useEffect / useMemo	localStorage
Dashboard	Filtering + derived metrics	codetrack_problems
Problem Library	CRUD + validation	codetrack_goals
Analytics	Streak + analytics calculations	codetrack_theme
Goals	Goal updates + export	JSON download backup

4.3 Execution flow

1. Browser loads the React entry point.
2. Initial records and goals are loaded from localStorage or the seeded dataset.
3. The root component computes derived values such as solved count, weekly minutes and streak.
4. The selected view renders its reusable child components.
5. User actions update React state through event handlers.
6. useEffect writes changed data back to localStorage.
7. Search/filter controls recompute the visible problem list.
8. Submitting the controlled modal form validates and inserts or updates a record.

5. FEATURES AND FUNCTIONALITIES

5.1 Dashboard

A practice-signal landing view summarizes the current streak, monthly target, weekly practice time, solved totals, bookmarks and average solve time. A recent-activity list gives one-click access to the latest problem records.

5.2 Problem Library

A table-oriented library exposes title, topic, date, difficulty, status, language and time. Search and filter controls narrow the dataset instantly. Records can be bookmarked, edited or deleted.

5.3 Problem logging

The modal form collects problem title, topic, language, difficulty, status, solve time, date, optional problem URL and notes. The same form supports create and edit operations.

5.4 Validation

A submission is rejected when the title or topic is empty, when solve time is outside the 1-600 minute range, or when an optional link does not begin with http:// or https://.

5.5 Analytics

The analytics view derives average solve time, seven-day solved output, difficulty mix and top practice topics from the current dataset.

5.6 Goals

The learner can set a monthly solved target and weekly targets for practice minutes and topic coverage. Progress bars and remaining values are calculated from stored records.

5.7 Local persistence

Problem records, goals and theme selection are written to localStorage through useEffect. The project does not require a backend for the classroom demonstration.

5.8 Utility features

A bookmark flag keeps review-worthy records visible, the theme toggle changes between dark and light modes, and JSON export creates a portable backup file.

5.9 Component map

Component	Responsibility
App	Global state, view selection, persistence, CRUD entry points
NavItem	Sidebar navigation control
Dashboard	Summary cards, target cards and recent records
Library	Search, filters and problem table
ProblemRow	Reusable problem summary row
Analytics	Practice metrics and charts
Goals	Target display and goal settings form
ProblemModal	Controlled create/edit form and validation

6. DATA MODEL AND IMPLEMENTATION

6.1 Problem record schema

Field	Type	Meaning
id	string	Unique problem record identifier
title	string	Programming problem title
topic	string	Primary algorithm/data-structure topic
language	string	Language used for the solution
difficulty	enum	Easy, Medium or Hard
status	enum	Solved, Review or Attempted
time	number	Approximate solve time in minutes
solvedOn	YYYY-MM-DD	Practice date
url	string	Optional problem reference link
notes	string	Approach, edge cases or review notes
bookmarked	boolean	Marks a record for quick revisit

6.2 Goal schema

Field	Type	Meaning
monthly	number	Target number of solved problems in the current month
weeklyMinutes	number	Target weekly practice time
weeklyTopics	number	Target number of distinct topics touched in the week

6.3 State management

The App component owns the shared problem list, goal values, current view and filter state. Child components receive data and callbacks through props. Immutable updates are performed with map, filter and object spread so React can detect changes predictably. useMemo is used for filtered lists and statistics so repeated calculations are tied to their relevant inputs rather than executed for unrelated changes.

6.4 Selected implementation excerpt

```
const solved = useMemo(() => problems.filter(p => p.status === "Solved"), [problems]);
const weeklyMinutes = useMemo(
  () => weekSolved.reduce((total, p) => total + Number(p.time || 0), 0),
  [weekSolved]
);

useEffect(() => {
  localStorage.setItem("codetrack_problems", JSON.stringify(problems));
}, [problems]);
```

7. USER INTERFACE - DASHBOARD AND LOGGING

The following figures represent the new CodeTrack interface designed for this implementation. The dashboard emphasizes coding-practice signals rather than habit completion. The fixed rail keeps the four views visible and the main surface uses compact cards for quick scanning.

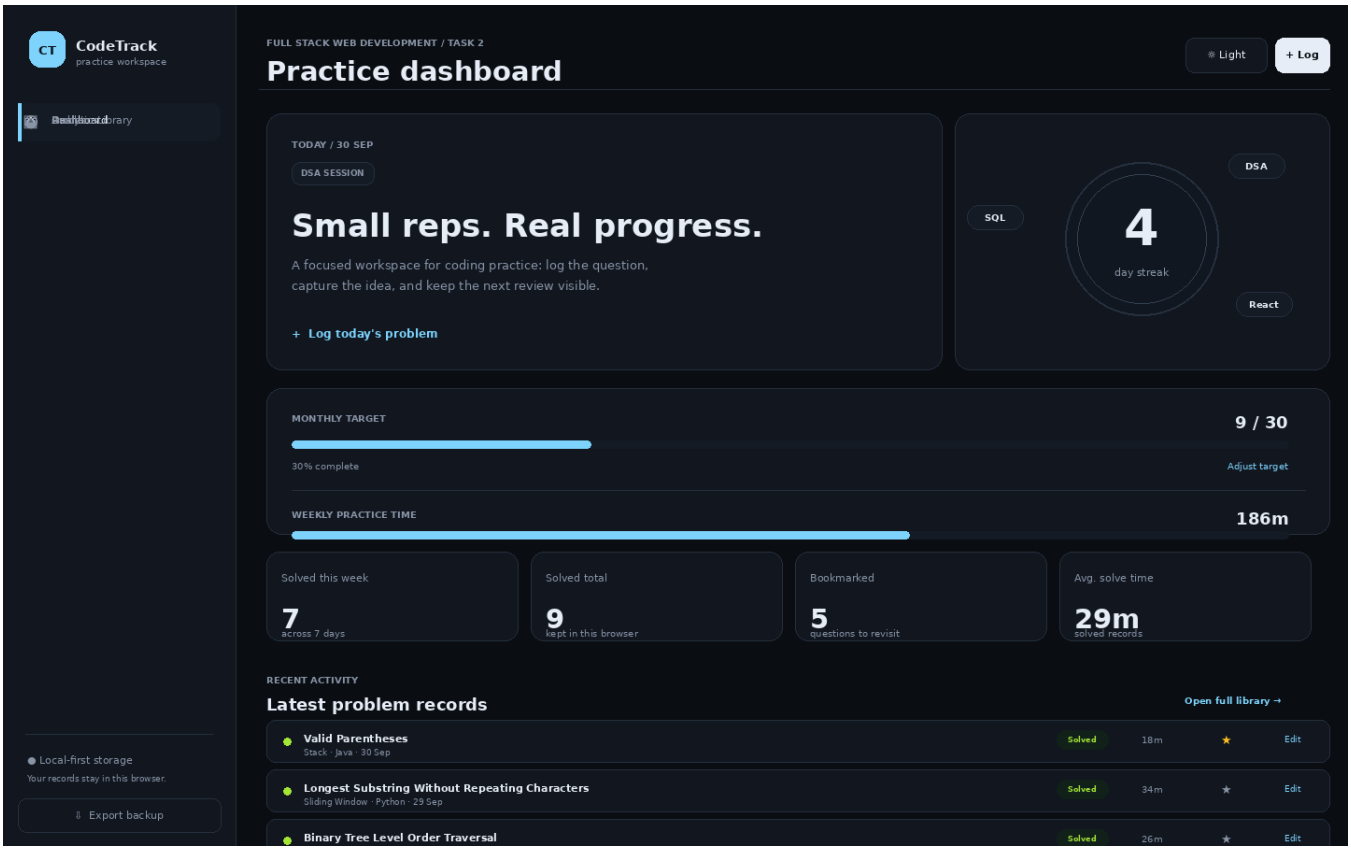


Figure 1. CodeTrack Dashboard - practice signal, targets, metrics and recent records

The logging workflow is intentionally separated into a modal so the learner can add a record without leaving the current view. The form captures enough context for later analytics while remaining small enough for rapid entry.

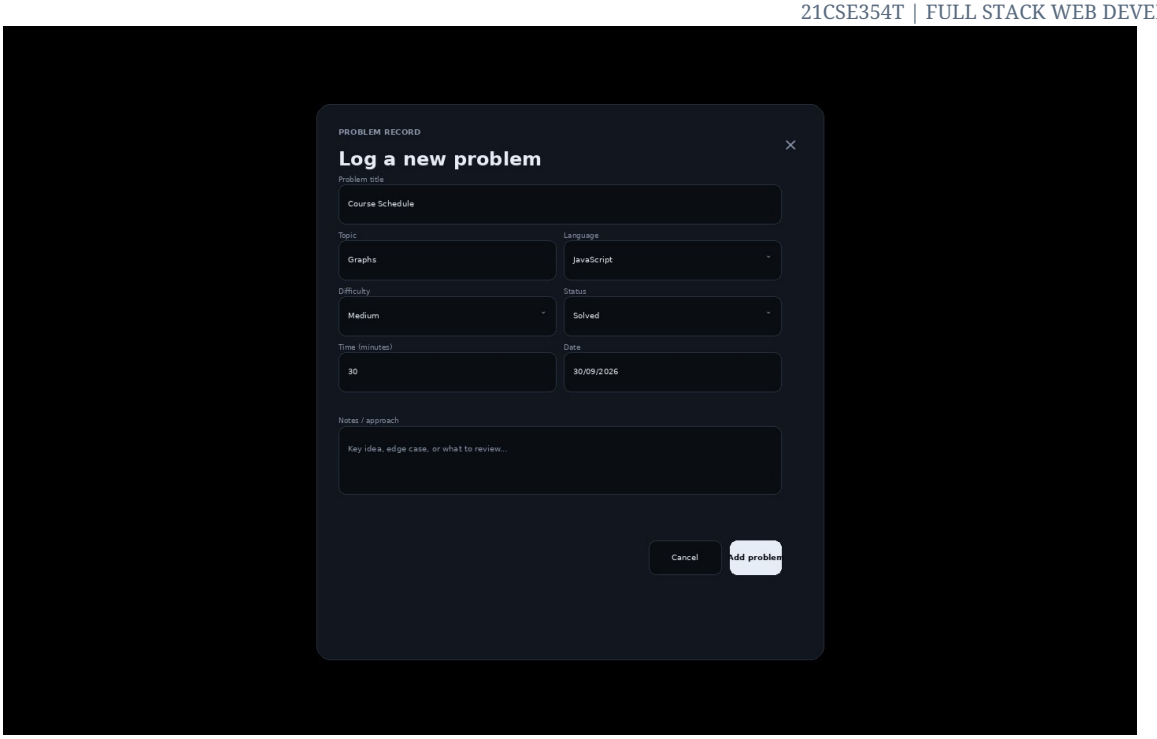


Figure 2. Log Problem modal - controlled data-entry form with validation fields

Interface design choices

- Left navigation rather than top segmented tabs
- Table-based library rather than habit cards
- Problem-specific labels and metrics
- Neutral dark dashboard with a blue/lime analytical accent
- One primary action: Log problem, available from every view

8. USER INTERFACE - LIBRARY AND ANALYTICS

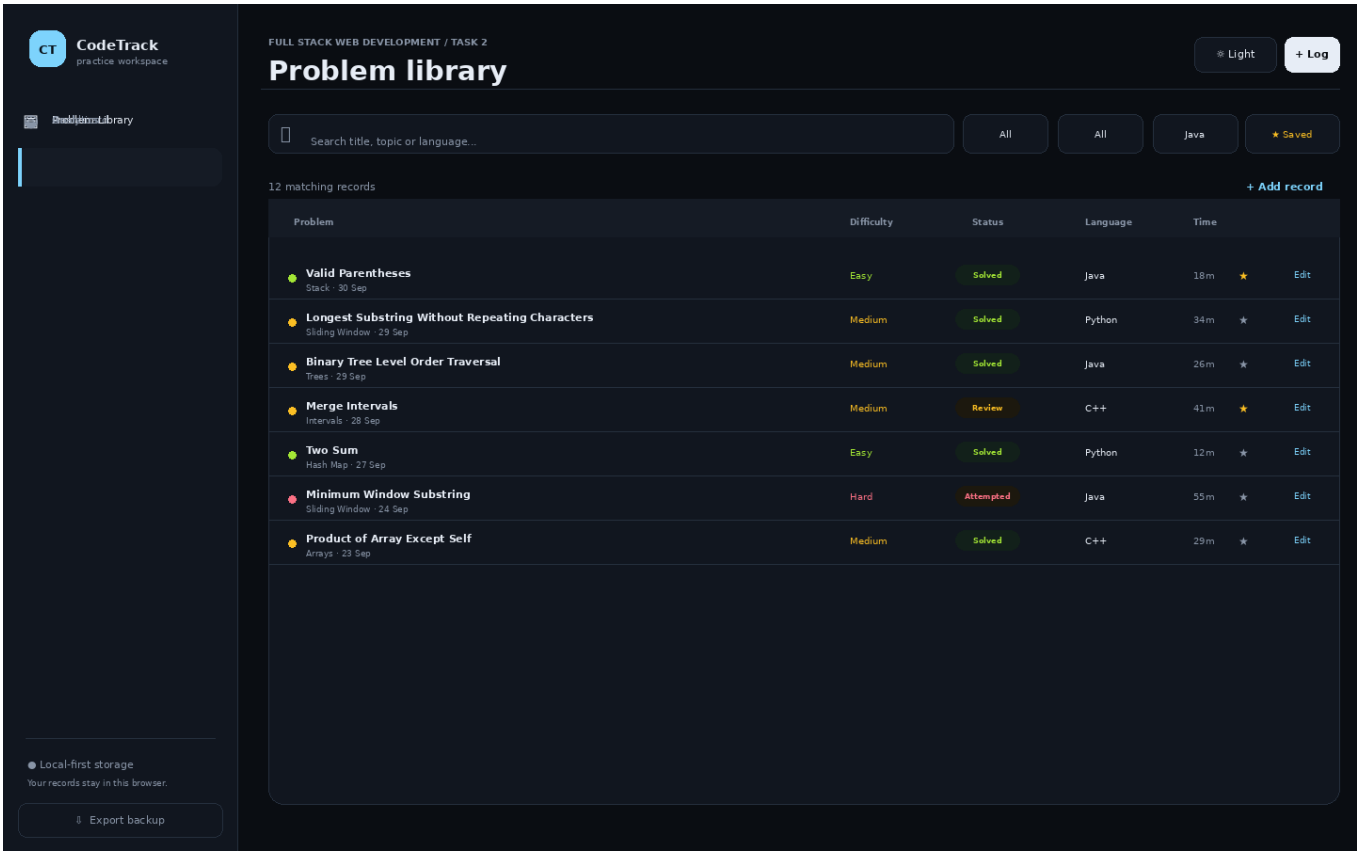


Figure 3. Problem Library - search, filters, bookmarks and record actions

The library is optimized for retrieval. Search terms are matched against title, topic and language, while dropdown filters can be combined. The bookmark control creates a lightweight review queue without introducing another database structure.

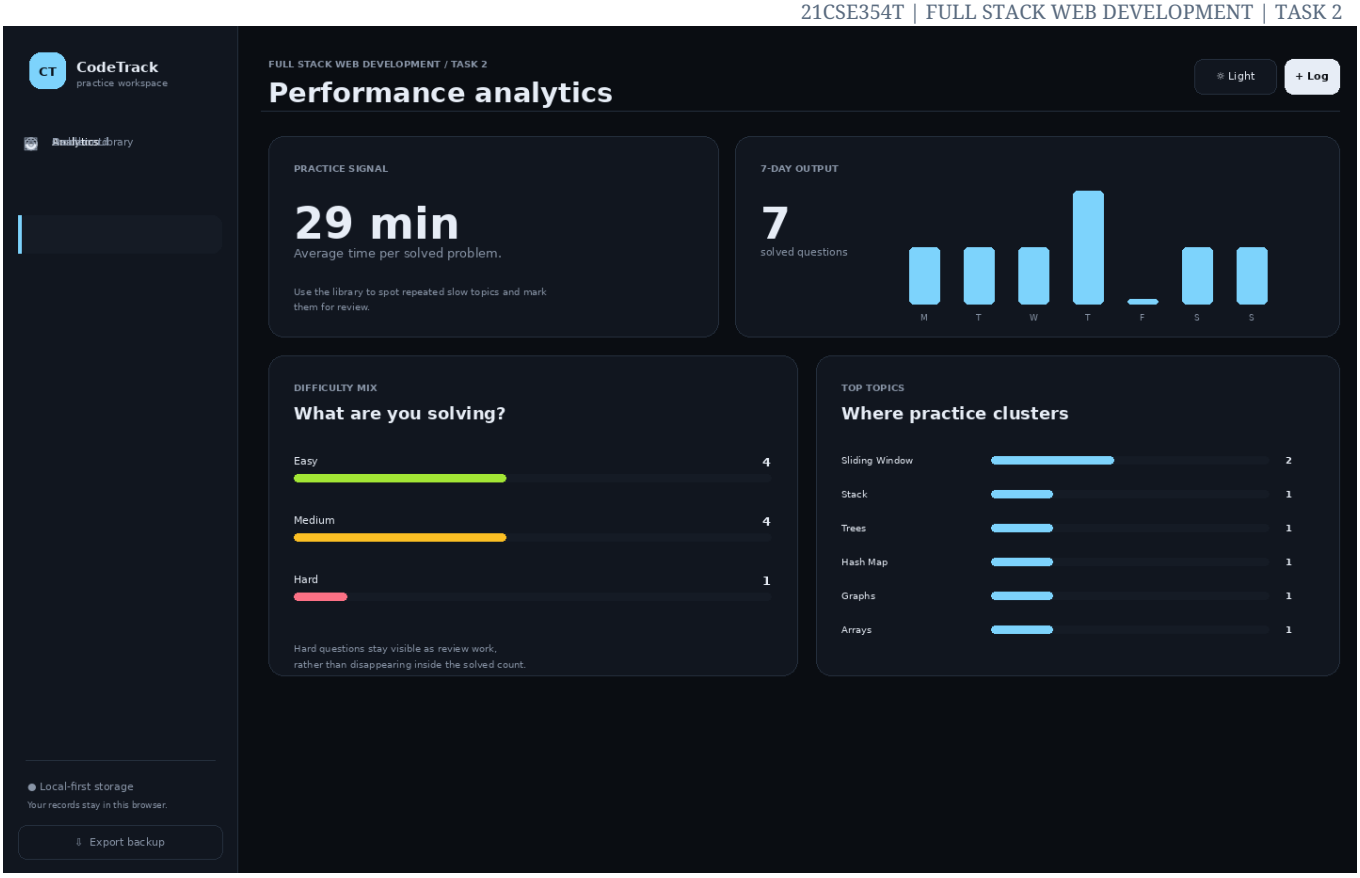


Figure 4. Performance Analytics - average solve time, seven-day output, difficulty mix and topic clusters

The analytics screen does not pretend to provide predictive intelligence. It deliberately shows transparent calculations that can be explained line-by-line during evaluation.

9. USER INTERFACE - GOALS AND RESPONSIVE BEHAVIOUR

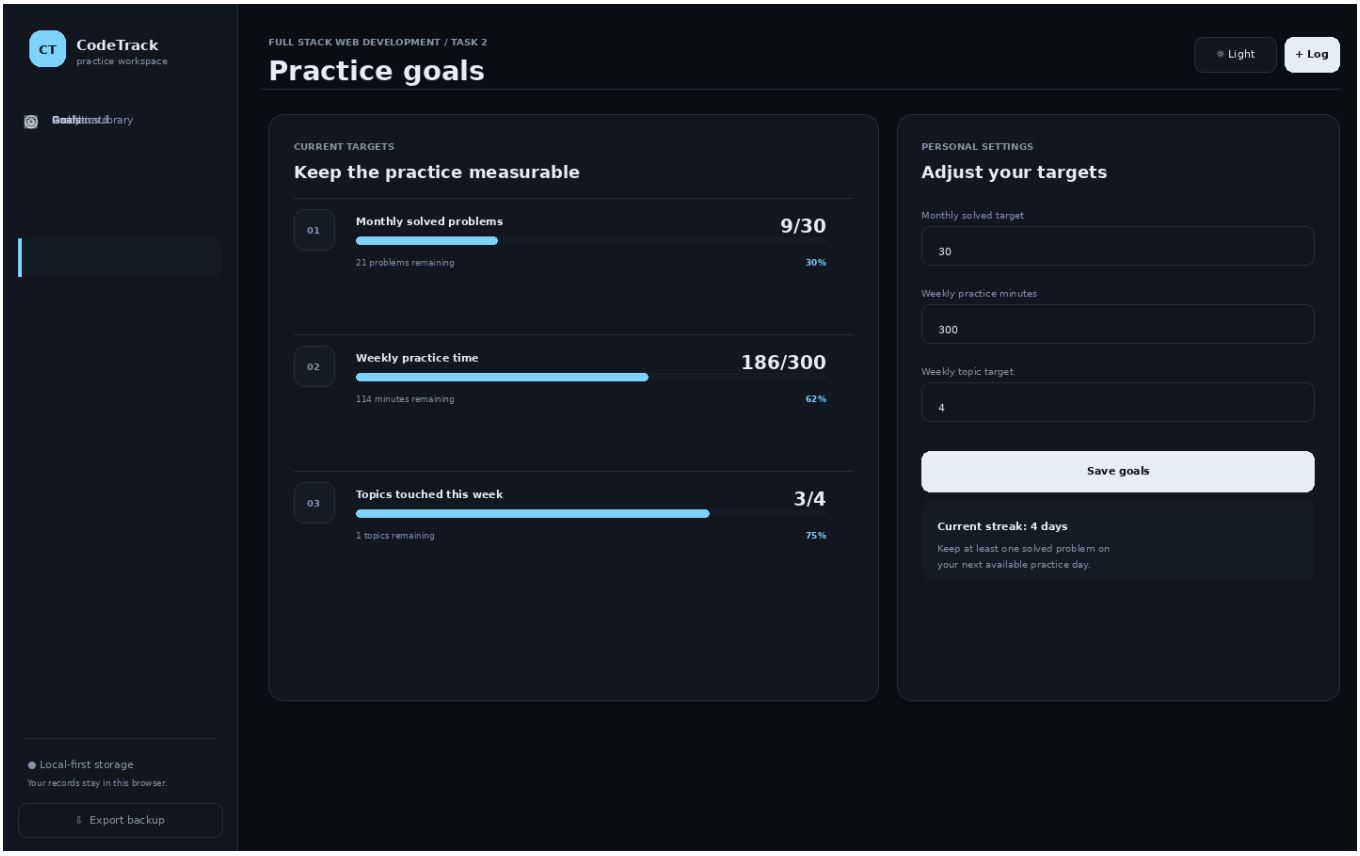


Figure 5. Practice Goals - configurable monthly, weekly-time and topic targets

9.1 Responsive strategy

- Desktop: persistent navigation rail with multi-column analytical cards.
- Tablet: columns collapse where required while preserving the dashboard hierarchy and table controls.
- Mobile: navigation compresses into compact buttons, the library shifts from a wide table to stacked records, and forms become single-column.

9.2 Accessibility-oriented details

- Form controls have visible labels.
- Buttons use text labels for primary actions instead of icon-only controls.
- Focus states are defined on inputs and selects.
- Content hierarchy uses clear headings, small uppercase section labels and consistent spacing.

10. TESTING AND OBSERVED RESULTS

Verification focused on source-level correctness and the functional scenarios required by the assignment. The React source was parsed with the TypeScript compiler in JSX mode without reported syntax/type-check errors for the included files; the remaining runtime scenarios are represented by the corresponding handlers and UI states in the project.

Test / Operation	Expected behaviour	Result
Application launch	Dashboard renders as the default view with seeded records	Implemented
View navigation	Dashboard, Library, Analytics and Goals switch without page reload	Implemented
Add problem	Valid form creates a new record immediately	Implemented
Validation	Empty title/topic, invalid time or malformed link are rejected	Implemented
Edit problem	Existing record updates without changing its identity	Implemented
Delete problem	Record is removed from the list and derived metrics recalculate	Implemented
Search	Title, topic and language queries narrow visible records	Implemented
Filtering	Difficulty, status, language and saved-only filters can be combined	Implemented
Bookmark	Saved state toggles for an individual record	Implemented
Analytics	Solved count, average time, difficulty and topic summaries recalculate from state	Implemented
Goals	Monthly, weekly-minute and topic targets update progress indicators	Implemented
Persistence	Problem data, goals and theme are stored using localStorage	Implemented
Responsive layout	CSS media queries reduce multi-column layouts on smaller widths	Implemented

10.1 Error-handling decisions

- Invalid modal submission is stopped before state mutation.
- Delete requires browser confirmation to reduce accidental record loss.
- Optional URL validation rejects non-http(s) strings.
- localStorage reads are wrapped in try/catch so malformed stored data falls back to the seed dataset.

11. LIMITATIONS, FUTURE ENHANCEMENTS AND CONCLUSION

11.1 Limitations

- Single-user local storage means records remain on one browser/device.
- The JSON export is a convenience backup, not encrypted storage.
- Analytics are descriptive rather than predictive and are intentionally lightweight.
- There is no server-side validation because the submission is a front-end React assignment.
- The project is deployment-ready but this package does not claim a live hosted URL.

11.2 Future enhancements

- Backend sync with Supabase or Firebase for multi-device persistence.
- Authentication and per-user private histories.
- Question-set imports from CSV/JSON and reusable topic templates.
- Longer-range charts, heat maps and time-per-topic trend analysis.
- Reminder scheduling and a session timer for timed practice.
- PWA support for offline installation and mobile home-screen access.

11.3 Conclusion

CodeTrack delivers a complete, independently structured solution to Task 2 Problem No. 43. It demonstrates React component design, hooks-based state management, controlled forms, validation, CRUD operations, filtering, derived calculations, responsive CSS and client-side persistence in one coherent application. The main learning value is the transformation of a raw problem log into a transparent practice dashboard: every metric is computed from the same records the learner enters, making the interface both useful and easy to explain during an academic evaluation.

12. REFERENCES

- React Documentation - https://react.dev/
- MDN Web Docs - JavaScript Guide and Array Methods - https://developer.mozilla.org/en-US/docs/Web/JavaScript/
- MDN Web Docs - CSS Layout and Responsive Design - https://developer.mozilla.org/en-US/docs/Learn/CSS/CSS_layout
- MDN Web Docs - Web Storage API - https://developer.mozilla.org/en-US/docs/Web/API/Web_Storage_API
- Vite Documentation - https://vite.dev/
- SRM Institute of Science and Technology - 21CSE354T Full Stack Web Development Task 2 problem statement and common requirements.

Originality note: This submission was written as a fresh implementation for AJAYAMSAVASAN ARULMURUGAN and uses a different problem statement, data model, information architecture, interface structure and feature set from the supplied Habit Tracker example. The supplied files were used only as assignment/format references.